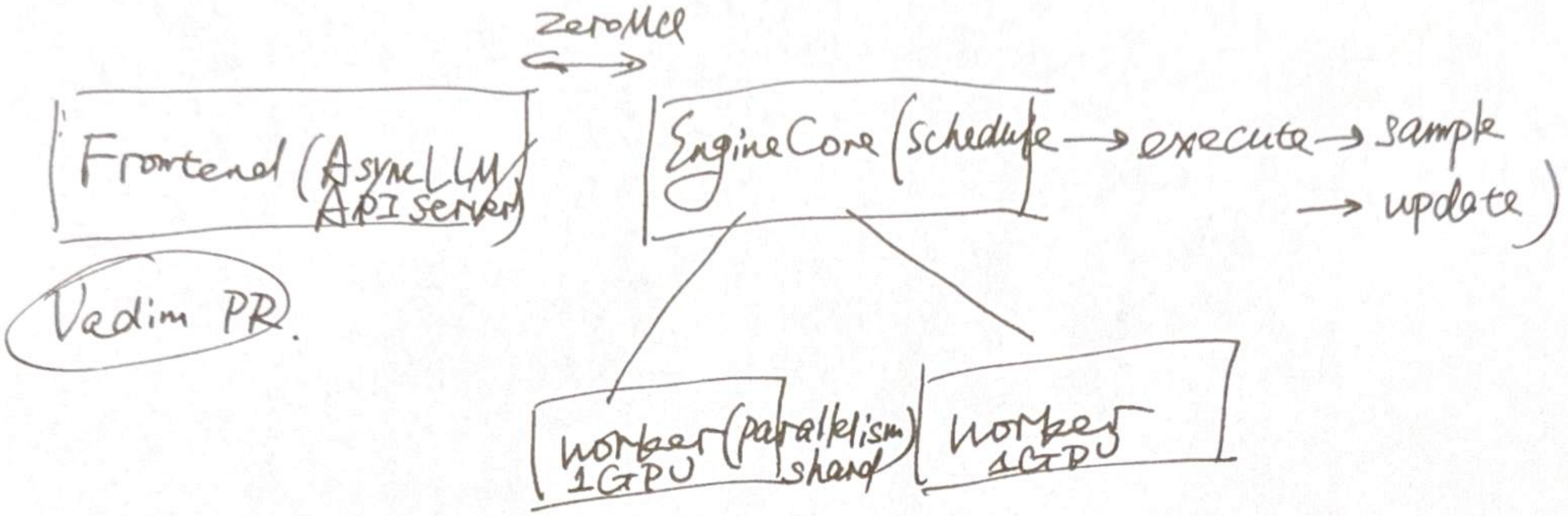
vllm serve 启动:



①

scheduler setup a token budget,
for the prefill, it based on the token budget,
to do the chunked prefill.

---

scheduler serve running requests first, then waiting requests (FIFO or priority)
如果当前 kv cache 不够 allocate to/for running request,
则 preemption, 其实就是本该从 waiting 里出来 into req, 基于时间已经给它 save kvcache spots, 也先改回 waiting, 为把/free 其 kvcache 给 running 为用.
Hacks.

---

prefill wants a chunked-scan kernel while decode wants a
复发的, 周期小的, 循环的.
recurrent step kernel.

就原本在 disaggregation PD 之前, flatten mixed batched 得 split apart,
at the model layer.
Vadim PR

KV cache ②

block table: key value

logical block idx ⟶ physical block idx

这其实是 PagedAttention in 论文, attention kernels gather K/V through the block table, so sequences don't need contiguous memory. so zero-fragmentation, GPU HBM 利用率高

block size by default 16 tokens/block.

问：如何 ↑ tokens/block ? pros: ① ↓ block table overhead, ② ↑ kernel efficiency. (∵ kernel 去读连续 block tasking)

cons: ① prefix hit ↓

② more fragmentation in GPU.

Vadim PR

第一阶段, one engine step, exe:

① scheduler ~~pick~~ call KV manager, pick token allocations for this step under the budget, 并 prefix caching, allocate/reuse KV blocks. 打包 为 SchedulerOutput.

② 'GPUModelRunner' builds inputs: flatten tokenIDs, positions, block tables.

③ Forward pass: (i) attention kernel (prefill) √ read/writes KV through block table.

(ii) For decode, for decode batches, execution replays CUDA graph (or eager).

④ Sampler: logits → (temprature, topk/p...) → sampled token IDs.

Sampling 也发生在 GPU 上，spec-decode in verification also happens on GPU.

① Scheduler update: append tokens, mark newly-full blocks (hash them, publish to prefix cache) ③

detect decode stop, free blocks of finished requests, 并 outputs stream back to frontend for detokenization.

注意四个点：

① prefix caching 查表点发生在 prefill 入口，但被命中的 KV 内容可以来自任何曾经 prefill 或 decode 的 block table.

② block table 全程都在，prefill 写，decode 读，finish 释放.

③ scheduler 决定 stop 基本都是 decode 步之后判 EOS/length/abort 的瞬发的，release 会将 block LRU 即非 clean.

Nccl kernel 里防改的传输不是 cudaMemcpy 到 host 再回来，而是 CUDA kernel 走这四种硬件直连通道：

NVLink/NVSwitch	同一个 node 的 GPU 之间
PCIe P2P	同 node 无 NVLink 的.
RDMA	跨节点.

Vaelim PR
all-reduce
Symm-mem

With DP + EP for MoE models, all DP ranks must step in ④
lockstep through the MoE all-to-alls, if one rank has
no requests, it runs dummy steps so collectives do NOT
deadlock. Vadim has PRs on exact this: ("Log dummy DP step",
"Publish request counts at the start of each engine step"
for the load balancer") ⛤

KV connector

Process 1: API server / frontend ← HTTP, tokenize, detokenize, Stream.
|(ZEROMQ)

Process 2: EngineCore
- Scheduler ← CPU only Python logic
  - KV cache manager ← (allocator, bookkeeper)
  - Connector (scheduler half) ← (planning only, no data)

(IPC to workers)

KV connector管两件事：① disagg-PD transfer ② KV offload/external caching

Process 3: GPU workers (one per GPU)
- GPUModelRunner ← forward pass, CUDA kernel
  - attention kernels ← READ/WRITE KV blocks
  - connector (worker half) ← execute transfer (NCCL/NIXL/copies)
- the KV cache tensors themselves ← actual GPU memory

Speculative decoding:

Decode is memory-bandwidth-bound: generating one token requires streaming all model weights + KV through HBM to do a small FLOPs per byte. The GPU's compute unit idle while weight stream. (target model)

If the big model can verify all K in one forward pass (like a mini prefill)

The small model stream fewer weights passes per token. (draft model)

Vadim 在这有个 E2E 的 PR for bug fix.

除此还有两个 PRs { "Fix accuracy bugs in Qwen3-Next MTP"
"force unquantized mtp.fc for MFP4".

接受 x 的概率 $= \min\left(1, \frac{P(x)}{Q(x)}\right)$

若 $P(x) < $Q(x), 则 draft 高估了这个 token, 只以 $\frac{P(x)}{Q(x)}$ 的概率接受 (<1)

大前提:

Draft $Q(x), 小模型, 快. $Q(x) 是它给 token $x 的概率.

Target $P(x), 大模型, 慢但准.

Draft 从 $Q(x) 里 sample 出一个候选 token $x.

Target 在验证 pass 里算出完整 $P 分布. 用 ___ 决定接受 or 拒绝.

拒绝后的补救：residual distribution.

残差 diviation

$$P_{residual}(y) = \frac{\max(0, P(y) - Q(y))}{\sum_z \max(0, P(z) - Q(z))}$$

核心证明：为什么输出分布精确等于 $P(x)$？

$P(\text{最终输出 } x) = P_A(x) + P_B(x)$

↙ 被 draft 采到且被接受

↘ draft 采到的 token 被拒，从残差 (residual) 中补出来的 x.

$P_A(x) = Q(x) \cdot \min\left(1, \frac{P(x)}{Q(x)}\right) = \min(Q(x), P(x))$

$P_B(x) = \cancel{\max(Q(x), P(x))}$ $\max(0, P(x) - Q(x))$

所以：$P = P_A(x) + P_B(x) = \min(Q(x), P(x)) + \cancel{\max(Q(x), P(x))}$ $\max(0, P(x) - Q(x))$

情况 1：$P(x) >= Q(x) \Rightarrow P = Q(x) + P(x) - Q(x) = P(x)$. ✓

情况 2：$P(x) < Q(x) \Rightarrow P = P(x) + 0 = P(x)$ ✓

所以，正确性与 draft 的质量无关.

Draft 质量只影响一千东西，接受率，也就是速度.

越小模型猜得越多准，能带来越多加速.

- Small size, accuracy-critical modules (drafters, routers/gates, norms) ⑦
another first things you exempt from quantization,
Vadim has a PR doing NVFP4 exempt for 'mlp.gate'.

- ☆ Spec decoding 盈利模型，在 memory-bound (TensorCore wait weight 从 HBM 搬过来) 阶段白嫖闲置 FLOPs.

Batch ↑ concurrency ↑，闲置 FLOPs 没那么多，且 compute bound ↑，Spec decoding 没那么好用了.

α = draft model 的接受率，一般 0.8 左右，K, 4-5 就可以.

CUDA graph 把一堆 kernel launch 提前 record，之后 replay 时，省下 kernel launch 的 CPU 开销 (10+μs/launch × 上百个 kernels)
Vanilla decode 时，每次都自下 stop 处理一个 token，这样 CUDA 很静态. 但 spec decoding 是动态的，你不知道会 accept 几个 tokens.

真实 tokens [t1, t2, t3, –, –] 只有3个有效

padded [t1, t2, t3, PAD, PAD]

mask [1, 1, 1, 0, 0]

这样虽然 CUDA graph 能用了，但 padding 的 FLOPs 是浪费的.

那，padding 有两种？
一种是一个 seqeme 内部的 padding，length 为 draft model 的 K 长度。
还有一种，是 batch 的，为 continuous batching 的

quantization 的好处：

① 少了之后 decode 在 transfer weight/KV to Tensor Core 的，加速！

② 更大的 batch size, context length, KV cache pool 更大.

value = $(-1)^{sign} \times 2^{exponent} \times (1 + mantissa\ fraction)$

Exponent bits (E): ~~越多越好~~ ↗ 范围大，精度差.

Mantissa bits (M) ↗ 精度高，范围小.

FP8 (E4M3) max = 448. ④

FP4 (E2M1) max = 6.0

Scale s 是一个乘数，用来把"小范围的量化整数/低精度浮点数"恢复回"原始高精度浮点数的量级".

你不在量化什么，exemption list:

① Norms (RMSNorm / LayerNorm)，用高精度计算（我的 PR 就是关 fused path 卷入的，RMSNorm 用哪个 dtype 做计算的问题.

② MoE router/gate (mlp.gate)，一个极小的 linear 层，决定 token 路由到哪些 expert. 会影响 expert 的选择.

③ MTP/draft heads (mtp.fc)，量化后噪声 → 接受率↓ → e2e latency↑

④ Embedding / lm_head，通常保留高精度.

CUDA graph

一个 decode step 是 N 个到上百个短 kernel. ① 每次 kernel launch 都 5~10 μs in CPU 开销, ② 再加上 batch, GPU 在 kernel 之间就是干等 CPU. (不同 serialize)

所以让连续 serial 在一次 capture 里跑一遍, stream capture 会将每个 kernel, 以及 dep 关系记录进一个 graph 对象里去.

graph 里, shape 是锁死 in. 动态 batch size 没法在同一张 graph 中变化。解决方法也是 padding, 多准备 N 个 batch size in graph, 当真正 requests 来了, 找最接近 in, padding.

如何 CUDA graph 也分 piecewise, attention in metadata (分段式 CUDA graph)

每步都在变, 而 GEMM 不变, 所以把 GEMM 拿去 graph, attention 留 eager mode.

Profiling & Benchmarking

Nsight system: timeline view — kernel launches, gaps, CPU threads, Nccl, memcpy.
Nsight Compute: single-kernel profill, Tensor core utilization, occupancy, memory, second step, once Nsight system tells you which kernel.

Q: How do you approach a perf problem? 答: nsys first to find where time goes, kernel time vs gaps vs sync, see if it is due to memory roofline, if it is NOT, then the fix is algo level, like fusion, layout, quantization.

My PRs vLLM so far 07/12/26

① [Spec Decode] Fix max_model_len logging in speculative config for draft model.

② [Bugfix] Guard against buffered token IDs in BaseThinkingReasoningParser closed streaming.

③ [Bugfix] Zero stale is_prefilling in padded CUDA graph rows for Mamba.

中文精讲：

一. 背景机制，三个前置概念：

1. persistent InputBatch（持久化批次）+ condense()

vLLM in model runner 不会每一步都重建 batch 的元数据，而是维护一个持久化 InputBatch：一组按"槽位(slot)"排列的数组，比如 num_computed_tokens[]，num_prompt_tokens[]，每个 active 请求 request 占一个 slot. 每步只做增量更新(diff)，避免每步重新构建、以及重新上传 CPU→GPU 的开销.

当有请求结束时，condense() 会做压实（compaction）：把最后面的活跃请求搬到刚空出来的低位槽（slot），让活跃请求始终占据数组的前缀（前 num_reqs slots）.

关键点：被腾空的尾部槽位不会被清零，里面还有旧数据，利用 num_reqs 知道前多少位置为 active reqs，后面的为 dead slots.

2. CUDA graph padding

CUDA graph 要求形状固定，所以 vLLM 按 bucket（合适的）拿 CUDA graph，batch=1,2,4,8.... 实际 batch 为 5 时，就填充(pad)到 8 来复用 batch=8 的 CUDA graph. padding row 用的正是那些 dead slots，也就是说，padding 把 dead slots 重新拉回了计算范围.

3. split_decode_and_prefills (decode/prefill 分流)

GDN/Mamba 一层有两个 kernel — prefill 走 chunked scan, decode 走 recurrent step. 所以每一步，混合 batch 必须被切分. 哪些行是 decode，哪些是 prefill，切分依据一个 bool 数组 is_prefilling，它的计算方式是逐行比较：

is_prefilling[i] = num_computed_tokens[i] < num_prompt_tokens[i]

二、Bug 的三重巧合，这个 bug 需要三个条件同时成立，缺一不可：

1. condense() 留下旧的脏数据.
2. padding 复活 dead slots
3. ~~Mamba 在 split_decodes_and_prefills 有个 parameter~~
3. padding 部分 is_prefill 为 true (可能).

Mamba 的 "decode-only CUDA graph replay can be classified as mixed decode/prefill metadata" ⑧.

⑭ [Bugfix] fix RMSNorm kernels to multiply in weight's native dtype.

依照 Python spec: $x = x.\text{to}(\text{weight.dtype}) * \text{weight}$

x: FP32; weight.dtype: toFP16; weight: FP16

⑮ [FlashAttn] Fix supports_kv_cache_dtype() accepting unhandled fp8 kv-cache dtype variants.

FlashAttentionBackend.supports_kv_cache_dtype(dtype) 这个 capability check 方法对一些它其实并不支持的 fp8 变体，也返回了 True，导致 backend selection 阶段选中了 FlashAttention，实际跑起来到 Kernel 里才报.

fp8 / fp8e4m3

fp8e5m2

⑥ [Bugfix] P2pNcclConnector: use stable external_req_id as NCCL closed
tensor key to fix disaggregation prefill hang.
P2pNcclConnector removed).

⑦ [Bugfix] Fix UBatchWrapper CUDA graph key to sum all ubatches, not just first two.
在大模型+MoE+多卡场景下，一次 forward 的时间被两类操作切分：
1. Compute：GEMM、attention、MLP、activation... 跑在 GPU SM 上.
2. Communication：TP 的 all-reduce、EP 的 all-to-all，跑在 Nccl/NIXL 上，走 NVLink/IB.
默认是串行的：
[compute layer i] → [all-to-all] → [compute expert] → [all-to-all]
ubatch 就是在[此时]交叉跑，避免 compute 的 SM idle 等 communication.

⑧ [Bugfix][Model] Qwen3-Omni: move cu_seqlens to GPU before VIT attention.
i) VIT：vision transformer (Omni)
ii) cu_seqlens 是在 CPU 上 init 的，传入 attention kernel 前要 move (copy) to GPU device.

⑨ 反问阶段：
① How does the team decide what lands upstream in vLLM vs what goes to internal? When you're optimizing for a new GPU generation, How to determine open source first or internal first?
②